SimpleGraph-main

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Graph
Programming Focus: General programming in c++

Simple Graph

For this computer assignment, you are to write a C++ program to implement several graph algorithms on simple graphs. These graphs are implemented with adjacency list representation. The `Graph` class is already defined in the header file `graph.h` and included as part of this repository.

The graph is created from a text file that contains the adjacency matrix representation of a graph. The first line of the file is an integer `n` indicating the number of vertices in the graph. Next there is a `(n+1)×(n+1)` table where the first row and the first column are names of vertices. The table records edges of the graph. Integer `1` indicates an edge exists between a pair of vertices. Integer `0` indicates no edge.

For a given graph `size`, each vertex of a graph can be referenced by index values `0, 1, …, ( size – 1 )`; however, vertices of a graph are labeled consecutively by capital letters, starting from `A`, which corresponds to the index value `0`. Use index values when you refer to a vertex in a graph in implementing the algorithms; use vertex labels only in printing. The edges are recorded in the data member `adj_list`, which is a `vector` object. The vertex labels are recorded in the data member `labels`, which is also a `vector` object. For example, you can use `labels[0]` to get the name of the first vertex, and `adj_list[0]` to get the `list` of edges (recorded as integer indexes of destination vertices) from the first vertex as the source.

The source file simplegraph.cc already contains the complete main function. Implement **ALL** member functions of the `Graph` class and put your implementations of these functions in **graph.cc**. (You can certainly insert inline code in the header file when the implementation of a member function only involves **at most** of couple of lines of code.)

Programming Notes:

Several functions are described in more details below:

- `Graph::Graph(const char* filename)` This is the constructor. It reads data in from the input file of the graph given in adjacency matrix representation and builds the graph with a adjacency list representation. This method sets the value of size, builds the vectors `labels` and `adj_list`. For example, for the following line of input: **D 0 1 0 0 1 0 0** Add edges to `adj_list[3]`, which records edges starting from vertex `D`, by adding values **1** and **4**, which are indexes for vertices `B` and `E`.

- `void Graph::Depthfirst(int v)` This private function is used to traverse a graph in the *depth-first traversal/search* algorithm starting at the vertex with the index value of `v`. To implement this method (and together with the `Traverse` method below), you may need several global variable and objects. For example, container objects to record the visiting order of all vertices, the container object to record the paths of traversing edges, and an integer indicating the current order in traversing.
- `void Graph::Traverse()` This public function is used to traverse a graph and invokes the above `Depthfirst` method. You will also need to display traverse result: the list of vertices in the order of their visit and the list of edges showing the path(s) of the traversal. At beginning of this method, you need to initialize the global variable(s) and object(s) used in `Depthfirst`.
- `void Graph::Print() const` This function prints the adjacency list for the graph. The following line is an example from an output: **D: B, E** It indicates there are edges from vertex D to vertices B and E.

Assignment Notes:

- Include any necessary headers and add necessary global constants.
- You are not allowed to use any I/O functions from the C library, such as scanf or printf. Instead, use the I/O functions from the C++ library, such as cin or cout.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (you need to construct and add Makefile) so that the TA only needs to invoke the command make to compile your source file and produce the executable file **simplegraph**. Make sure you use exactly the same file names specified here, i.e. **simplegraph**, in your Makefile, otherwise your submission will get 0 points. **Do not** change any existing code, including class definition and the main routine in the original files graph.h and simplegraph.cc. You can insert new code into graph.h and graph.cc, including data members to the header file if necessary. Usually the source file (.cc) contains the implementation of member methods of the class. You can add inline code in the header file (.h) when the implementation is very brief, containing just one or two lines for a member method. To test your program, execute `./simplegraph simplegraph.input &> simplegraph.out`, which will put the output (including any error messages) in file `simplegraph.out`. You can find the correct output of this program in file `simplegraph.refout`.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.